QuickBite — Unified UML 2.5 Class Diagram Specification

Complete Structural Model: Generalization, Realization, Composition, Aggregation, & Association

Standard: OMG UML 2.5 / ISO-IEC 19505	**Platform:** Spring Boot 3 + Supabase + React OOP	**Diagram Scope:** 17 Entities + 3 MappedSuperclasses + 2 Interfaces + Gateways	**Version:** 2.0 (Complete Unified)

Comprehensive System Model: This document provides the complete, authoritative UML Class Diagram for QuickBite. It incorporates all 20 persistent classes, their exact fields, database constraints, business methods, lifecycle events, and relationship connectors (Generalization, Realization, Composition, Aggregation, Directed Association, and Dependency).

1. Formal UML 2.5 Notation Standard & Arrowhead Reference

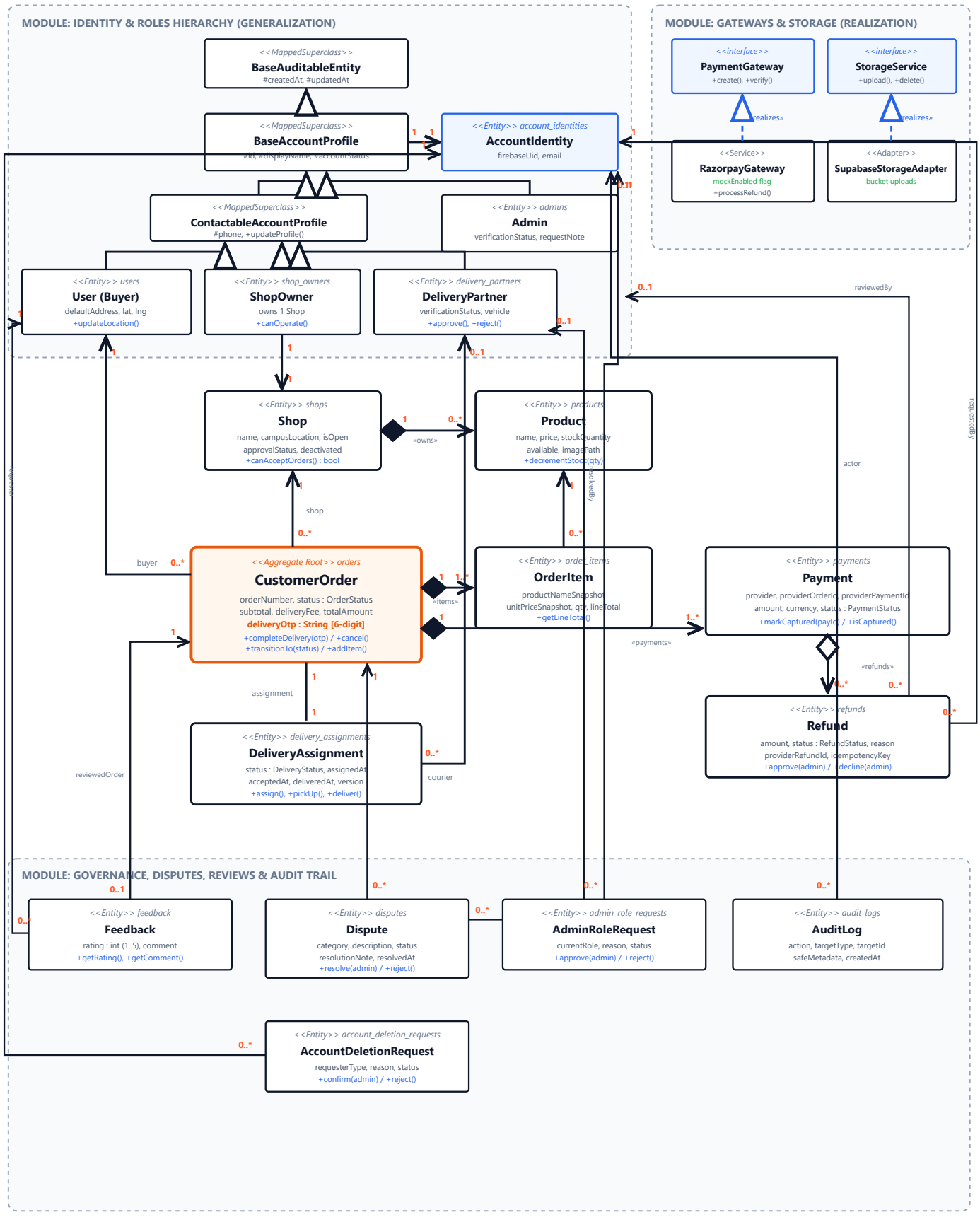
Every arrow and connector on QuickBite UML diagrams strictly follows formal OMG UML 2.5 graphical specifications:

Connector Type	Official UML Graphic Symbol	Mermaid	Lifecycle & Conceptual Semantics	QuickBite Implementation
Generalization (Inheritance)	Solid line + Hollow white triangle	`<|--`	**Is-a** hierarchy. Subclasses inherit all state, operations, and pre-persist callbacks.	Java `extends` `User extends ContactableAccountProfile`
Realization (Implementation)	Dashed line + Hollow white triangle	`<|..`	**Contract implementation**. Concrete class provides implementation for interface methods.	Java `implements` `RazorpayGateway implements PaymentGateway` `SupabaseStorageAdapter implements StorageService`
Composition (Strong Whole-Part)	Solid black diamond at Whole + Solid line	`*--`	**Exclusive lifetime ownership**. Part cannot exist without the Whole. Deletion cascades.	`@OneToMany(cascade = ALL, orphanRemoval = true)` `CustomerOrder *-- OrderItem` `Shop *-- Product`
Aggregation (Shared Whole-Part)	Hollow white diamond at Whole + Solid line	`o--`	**Container-member**. Whole references parts, but parts have independent lifecycles.	Separate table & review workflow `Payment o-- Refund`
Directed Association	Solid line + Open arrowhead pointing to target	`-->`	**Navigable reference**. Source holds a foreign key to the target entity.	`@ManyToOne` or `@OneToOne` `CustomerOrder --> User` `BaseAccountProfile --> AccountIdentity`
Bidirectional Association	Plain solid line with multiplicities	`--`	**Reciprocal 1:1 relationship**. Mutual navigation.	`CustomerOrder 1 -- 1 DeliveryAssignment`
Dependency	Dashed line + Open arrowhead	`..>`	**Uses-a transient dependency** (method parameter, invocation, calculation helper).	Service injection / DTO usage `OrderService ..> PricingService` `CheckoutPaymentService ..> PaymentGateway`

2. Complete Master Unified System Class Diagram

This full-scale vector UML diagram illustrates all 17 JPA Entities, 3 MappedSuperclasses, 2 Service Interfaces, Gateways, and their exact structural relationships:

FIGURE 2.1: QuickBite Architecture Master Class Diagram (All Domains Unified)



3. Complete Domain Enumerations

All 10 enum types governing system lifecycle state transitions across account eligibility, orders, courier transit, payments, and disputes:

Enumeration Type	Complete Set of Literal Values	Applied Domain Entity / Context
`AccountStatus`	`ACTIVE`, `SUSPENDED`, `DEACTIVATED`	`BaseAccountProfile` (All profiles)
`AccountTypeSnapshot`	`BUYER`, `SHOP_OWNER`, `DELIVERY_PARTNER`, `ADMIN`	Firebase Claims, Deletions, Role Requests
`VerificationStatus`	`PENDING`, `APPROVED`, `REJECTED`, `SUSPENDED`	`DeliveryPartner`, `Admin`, Requests
`ShopStatus`	`PENDING`, `APPROVED`, `REJECTED`, `SUSPENDED`	`Shop` (Approval state)
`OrderStatus`	`PLACED`, `ACCEPTED`, `REJECTED`, `PREPARING`, `READY`, `OUT_FOR_DELIVERY`, `DELIVERED`, `CANCELLED`	`CustomerOrder` (Lifecycle machine)
`DeliveryStatus`	`UNASSIGNED`, `ASSIGNED`, `ACCEPTED`, `PICKED_UP`, `DELIVERED`, `FAILED`, `CANCELLED`	`DeliveryAssignment` (Transit state)
`PaymentStatus`	`CREATED`, `PENDING`, `AUTHORIZED`, `CAPTURED`, `FAILED`, `REFUND_PENDING`, `REFUNDED`	`Payment` (Transaction state)
`RefundStatus`	`REQUESTED`, `APPROVED`, `DECLINED`, `FAILED`	`Refund` (Disbursement review)
`DisputeStatus`	`OPEN`, `UNDER_REVIEW`, `RESOLVED`, `REJECTED`	`Dispute` (Mediation state)
`DeletionRequestStatus`	`PENDING`, `APPROVED`, `REJECTED`	`AccountDeletionRequest`

4. Full Master Mermaid UML Source Code

Copy and paste the code below directly into Mermaid Live Editor (`mermaid.live`), Notion, Obsidian, GitHub markdown, or Visual Studio Code:

```
classDiagram
    direction TB

    %% ========================================================================
    %% 1. GENERALIZATION / INHERITANCE (<|--) [Solid line + Hollow triangle]
    %% ========================================================================
    BaseAuditableEntity <|-- BaseAccountProfile : extends
    BaseAccountProfile <|-- ContactableAccountProfile : extends
    BaseAccountProfile <|-- Admin : extends
    ContactableAccountProfile <|-- User : extends
    ContactableAccountProfile <|-- ShopOwner : extends
    ContactableAccountProfile <|-- DeliveryPartner : extends

    %% ========================================================================
    %% 2. REALIZATION / INTERFACE IMPLEMENTATION (<|..) [Dashed + Hollow triangle]
    %% ========================================================================
    PaymentGateway <|.. RazorpayGateway : realizes
    StorageService <|.. SupabaseStorageAdapter : realizes

    %% ========================================================================
    %% 3. COMPOSITION (*--) [Solid filled diamond on whole + Cascade lifecycle]
    %% ========================================================================
    Shop "1" *-- "0..*" Product : contains
    CustomerOrder "1" *-- "1..*" OrderItem : orderItems
    CustomerOrder "1" *-- "1..*" Payment : paymentRecords

    %% ========================================================================
    %% 4. AGGREGATION (o--) [Hollow diamond on whole + Independent lifecycle]
    %% ========================================================================
    Payment "1" o-- "0..*" Refund : eligibleRefunds

    %% ========================================================================
    %% 5. DIRECTED & BIDIRECTIONAL ASSOCIATIONS (--> and --)
    %% ========================================================================
    BaseAccountProfile "1" --> "1" AccountIdentity : identity
    ShopOwner "1" --> "1" Shop : operates
    CustomerOrder "0..*" --> "1" User : buyer
    CustomerOrder "0..*" --> "1" Shop : shop
    OrderItem "0..*" --> "1" Product : catalogItem
    CustomerOrder "1" -- "1" DeliveryAssignment : dispatchAssignment
    DeliveryAssignment "0..*" --> "0..1" DeliveryPartner : assignedCourier
    Refund "0..*" --> "1" AccountIdentity : requestedBy
    Refund "0..*" --> "0..1" Admin : reviewedBy
    Feedback "0..1" --> "1" CustomerOrder : reviewedOrder
    Feedback "0..*" --> "1" User : author
    Dispute "0..*" --> "1" CustomerOrder : disputedOrder
    Dispute "0..*" --> "1" AccountIdentity : raisedBy
    Dispute "0..*" --> "0..1" Admin : arbitrator
    AdminRoleRequest "0..*" --> "1" AccountIdentity : applicant
    AdminRoleRequest "0..*" --> "0..1" Admin : reviewer
    AccountDeletionRequest "0..*" --> "1" AccountIdentity : requester
    AccountDeletionRequest "0..*" --> "0..1" Admin : reviewer
    AuditLog "0..*" --> "0..1" AccountIdentity : actor

    %% ========================================================================
    %% 6. DEPENDENCIES (..>) [Dashed line + Open arrowhead]
    %% ========================================================================
    CheckoutPaymentService ..> PaymentGateway : uses
    CheckoutPaymentService ..> OrderService : delegates
    OrderService ..> PricingService : calculatesPricing

    %% ========================================================================
    %% CLASS DEFINITIONS WITH ATTRIBUTES & STEREOTYPES
    %% ========================================================================
    class PaymentGateway {
        <<interface>>
        +createOrder(BigDecimal, String, String)* GatewayOrder
        +verifyPaymentSignature(String, String, String)* boolean
        +processRefund(String, BigDecimal, String)* GatewayRefund
    }

    class RazorpayGateway {
        -String keyId
        -String keySecret
        -boolean mockEnabled
        +createOrder(BigDecimal, String, String) GatewayOrder
        +verifyPaymentSignature(String, String, String) boolean
        +processRefund(String, BigDecimal, String) GatewayRefund
    }

    class BaseAuditableEntity {
        <<MappedSuperclass>>
        #Instant createdAt
        #Instant updatedAt
        +getCreatedAt() Instant
        +getUpdatedAt() Instant
    }

    class BaseAccountProfile {
        <<MappedSuperclass>>
        #Long id
        #String displayName
        #AccountStatus accountStatus
        #Instant deactivatedAt
```

```
        +deactivate() void
        +suspend() void
        +reactivate() void
        +canOperateOnPlatform()* boolean
    }

    class ContactableAccountProfile {
        <<MappedSuperclass>>
        #String phone
        +updateProfile(String, String) void
    }

    class AccountIdentity {
        +Long id
        +String firebaseUid
        +String email
        +updateFirebaseUid(String) void
        +updateEmail(String) void
    }

    class User {
        +String defaultAddress
        +Double defaultLatitude
        +Double defaultLongitude
        +String foodInterests
        +updateDeliveryLocation(String, Double, Double) void
        +updateFoodInterests(String) void
    }

    class ShopOwner

    class DeliveryPartner {
        +VerificationStatus verificationStatus
        +String vehicleDetails
        +String serviceArea
        +approveVerification() void
        +rejectVerification() void
        +updateCourierDetails(String, String) void
    }

    class Admin {
        +VerificationStatus verificationStatus
        +String requestNote
        +approveVerification() void
        +rejectVerification() void
    }

    class Shop {
        +Long id
        +String name
        +String campusLocation
        +ShopStatus approvalStatus
        +boolean open
        +boolean deactivated
        +openShop() void
        +closeShop() void
        +canAcceptOrders() boolean
    }

    class Product {
        +Long id
        +String name
        +BigDecimal price
        +int stockQuantity
        +boolean available
        +decrementStock(int) void
        +restoreStock(int) void
    }

    class CustomerOrder {
        +Long id
        +String orderNumber
        +OrderStatus status
        +BigDecimal subtotal
        +BigDecimal deliveryFee
        +BigDecimal totalAmount
        +String deliveryOtp
        +transitionTo(OrderStatus) void
        +completeDelivery(String) void
        +cancel(String) void
    }

    class OrderItem {
        +Long id
        +String productNameSnapshot
        +BigDecimal unitPriceSnapshot
        +int quantity
        +BigDecimal lineTotal
    }

    class DeliveryAssignment {
        +Long id
        +DeliveryStatus status
        +Instant assignedAt
        +Instant deliveredAt
```

```
        +Long version
        +assign(DeliveryPartner) void
        +markPickedUp() void
        +markDelivered() void
    }

    class Payment {
        +Long id
        +String provider
        +String providerOrderId
        +BigDecimal amount
        +PaymentStatus status
        +markCaptured(String) void
        +isCaptured() boolean
    }

    class Refund {
        +Long id
        +BigDecimal amount
        +RefundStatus status
        +String reason
        +approve(Admin, String) void
        +decline(Admin, String) void
    }
```